

A Summer Internship Report

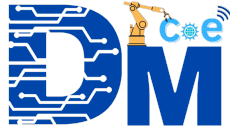
Submitted by

**Nandan Patel
Nirjari Bhatt
Saksham Mahajan**

**Center Of Excellence, Digital Manufacturing
Birla Vishvakarma Mahavidyalaya
Vallabh Vidyanagar**

**The Charutar Vidya Mandal (CVM) University
Vallabh Vidyanagar**

August 2025



CERTIFICATE

This is to certify that **Mr. Nandan Patel**, a student of **G. H. Patel College of Engineering**, has successfully completed the internship at **Center of Excellence, Digital Manufacturing** from **12/06/2025** to **06/08/2025**.

During the internship period, he has worked on *Development of Interactive Web Portal with Chatbot for the Center of Excellence in Digital Manufacturing* and has shown dedication, and a keen interest in learning.

This certificate is issued as a recognition of his successful completion of the internship program at **Center of Excellence, Digital Manufacturing**.

Dr. Vatsal Shah
Associate Professor,
IT Department, BVM

Dr. Vinay Patel
Professor and Principal,
BVM

Date:

Place: Vallabh Vidyanagar

Acknowledgement

I would like to express my sincere gratitude to **Dr. Vatsal Shah, Associate Professor, IT Department, BVM**, and **Dr. Vinay Patel, Professor and Principal, BVM**, for their valuable guidance, encouragement, and constant support throughout the course of my internship at the **Center of Excellence, Digital Manufacturing**.

Their mentorship, insightful suggestions, and timely feedback played a vital role in the successful completion of my project on “*Development of Interactive Web Portal with Chatbot for the Center of Excellence in Digital Manufacturing*”.

I am also thankful to the **Center of Excellence, Digital Manufacturing** for providing me with the opportunity, resources, and an excellent learning environment during this internship.

Finally, I extend my heartfelt thanks to my college, **G. H. Patel College of Engineering**, for giving me the platform to take up this internship and broaden my practical knowledge.

Mr. Nandan Patel

Abstract

The project titled “*Development of Interactive Web Portal with Chatbot for the Center of Excellence in Digital Manufacturing*” focuses on building an interactive and scalable digital platform for the **Center of Excellence in Digital Manufacturing (CoE-DM), BVM**. The portal serves as a centralized medium to showcase the center’s infrastructure, ongoing projects, research activities, and contributors, while providing an engaging and user-friendly interface.

A key feature of the platform is an **intelligent chatbot system**, designed to enhance user interaction and accessibility of information. The chatbot was initially implemented with intent-based responses and later upgraded to a **retrieval-augmented generation (RAG)** approach, enabling more contextual, accurate, and dynamic responses to user queries. This ensures that the chatbot can effectively address diverse informational needs while maintaining flexibility for future improvements.

The web portal was developed using **React.js/Next.js** with **Tailwind CSS** for a responsive and modern design, while **Firebase** was used for backend services and integration. The internship was structured in three phases:

- **Phase 1:** Training on Industry 4.0 concepts such as Industrial IoT, AR, Automation, and Predictive Maintenance.
- **Phase 2:** Dataset preparation, exploration of machine learning concepts, and chatbot system implementation.
- **Phase 3:** Full-stack development of the live project, including chatbot integration, web design, and deployment.

The outcome of this project is a **responsive, scalable, and interactive portal** that not only improves the digital presence of CoE-DM but also lays the foundation for advanced features such as an admin panel, cloud storage integration, and extended chatbot capabilities in the future.

LIST OF FIGURES

Figure 1: System Architecture	8
Figure 2: RAG Architecture	8
Figure 3: Database collections	14
Figure 4: Frontend snapshots	15
Figure 5: Admin panel	16
Figure 6: Chatbot inference	17

LIST OF TABLES

Table 1: Backend Technologies	4
Table 2: Frontend Technologies	6
Table 3: Tasks Performed	7

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION.....	1
1.1 OVERVIEW.....	1
1.2 OBJECTIVE AND SCOPE.....	1
 CHAPTER 2: TECHNOLOGY STACK.....	 3
2.1 OVERVIEW.....	3
2.2 BACKEND TECHNOLOGIES.....	3
2.3 FRONTEND TECHNOLOGIES.....	5
 CHAPTER 3: SYSTEM DESIGN AND IMPLEMENTATION.....	 7
3.1 SYSTEM OVERVIEW.....	7
3.2 TASKS PERFORMED.....	7
3.3 SYSTEM ARCHITECTURE.....	8
3.4 MODULE DESIGN.....	8
 CHAPTER 4: METHODOLOGY AND ALGORITHM.....	 9
4.1 INTRODUCTION.....	9
4.2 METHODOLOGY.....	9
4.3 ALGORITHM AND AI MODELS.....	11
 CHAPTER 5: SYSTEM SNAPSHOTS.....	 14
5.1 DATABASE INTERFACE.....	14
5.2 FRONTEND USER INTERFACE.....	15
5.3 ADMIN PANEL.....	16

5.4 CHATBOT INTERFACE.....	17
<i>CHAPTER 6: CONCLUSION</i>	18
CHAPTER 7: REFERENCES.....	19

CHAPTER 1: INTRODUCTION

1.1 OVERVIEW

The summer internship project titled *“Development of Interactive Web Portal with Chatbot for the Center of Excellence in Digital Manufacturing”* was undertaken to apply academic learning in a real-world technical environment. Conducted in collaboration with the Center of Excellence in Digital Manufacturing, the internship was structured in three phases, training on Industry 4.0 concepts, implementation of machine learning models, and live project development.

The project aimed to design a dynamic web portal to showcase the center’s infrastructure, projects, faculty, and student involvement, while integrating an AI-powered chatbot for interactive user engagement. The portal was developed using the MERN stack, with an admin panel to dynamically manage interns and faculty details, ensuring scalability and ease of content management.

1.2 OBJECTIVE AND SCOPE

1.2.1 Objectives

The primary objective is to design and implement a dynamic, user-friendly, and scalable platform that:

1. To design and develop a dynamic, responsive, and scalable web portal for the Center of Excellence in Digital Manufacturing using the MERN stack, with dedicated modules for stations (MIRAC-PC, TRIAC-PC, Assembly, Inspection, Performance Measurement, AS/RS), projects, publications, and faculty/student involvement.
2. To implement a Retrieval-Augmented Generation (RAG)-based chatbot, capable of providing context-aware, accurate, and interactive responses to user queries, assisting in navigation, and enhancing accessibility of the center’s resources.
3. To integrate an admin panel for authorized users to dynamically manage content such as interns, faculty, and project details without external dependencies.
4. To ensure cross-platform compatibility, performance optimization, and SEO readiness, enabling fast, secure, and device-agnostic access to the portal.
5. To provide comprehensive documentation and deployment guidelines, ensuring maintainability, scalability, and a roadmap for future enhancements such as cloud storage and extended chatbot capabilities.

1.2.2 Scope

The scope of the project includes:

1. **Interactive Web Portal Development:** Designing a modern, responsive UI with dedicated sections for stations, projects, faculty, students, publications, and achievements, along with optional news and updates.
2. **Admin Panel Functionality:** Providing authorized users with an interface to dynamically add, update, or manage information such as projects, interns, and faculty members.
3. **Chatbot Integration:** Embedding a conversational assistant to answer FAQs, explain station functionalities, assist with navigation, and recommend relevant content through a floating widget.
4. **Cross-Platform Responsiveness:** Ensuring smooth accessibility and usability across desktops, tablets, and smartphones.
5. **Performance & Optimization:** Implementing SEO best practices, structured content, and fast-loading web pages to enhance visibility and user experience.
6. **Documentation & Handover:** Delivering source code, deployment instructions, maintenance guidelines, and a roadmap for future improvements and scalability.

CHAPTER 2: TECHNOLOGY STACK

2.1 OVERVIEW

The development of the project required the integration of modern web technologies, database systems, and AI frameworks. The chosen technology stack ensured scalability, responsiveness, efficient data management, and intelligent user interaction.

The web portal was built using the MERN stack (MongoDB, Express.js, React.js, Node.js), which provided a robust full-stack development environment for designing a responsive and dynamic user interface along with a secure and flexible backend. An admin panel was implemented within the same stack to enable dynamic content management for interns, faculty, and project details.

For the chatbot, a Retrieval-Augmented Generation (RAG) approach was adopted, combining large language models with a dense retrieval mechanism to deliver accurate, context-aware responses. This significantly enhanced the user experience by enabling more intelligent, domain-specific conversations compared to traditional intent-based chatbots.

2.2 BACKEND TECHNOLOGIES

The backend of the project was designed to handle dynamic content management, API communication, and intelligent conversational processing. It was developed using Node.js with Express.js for the web services and MongoDB for database operations. In addition, the chatbot functionality evolved across phases—from rule-based frameworks to large language models, and finally to a Retrieval-Augmented Generation (RAG) approach.

2.2.1 Node.js and Express.js

The backend was built using **Node.js**, with **Express.js** as the web framework to develop RESTful APIs. These APIs enabled seamless integration with the frontend and supported dynamic management of content such as faculty, students, projects, and station details. Key features included:

- Fast and scalable communication with the MongoDB database.
- Custom routing for efficient content delivery.
- Middleware support for authentication and validation.

2.2.2 Database: MongoDB

The system utilized MongoDB, a NoSQL database, to store and manage structured and unstructured content dynamically. Collections were designed for:

- Faculty and students

- Projects and publications
- Stations and infrastructure details
- Admin authentication

MongoDB provided flexibility, scalability, and fast query performance, supporting real-time updates to the web portal.

2.2.3 Chatbot Backends

Phase 1: Rasa (Open Source NLP Framework)

- **Framework:** Rasa Open Source
- **Pipeline Tools:**
 - SpaCy for tokenization
 - DIETClassifier for intent classification and entity extraction
- **Policies:** Fallback policy, TEDPolicy, and memoization defined in config.yml
- **Limitation:** Heavy dependence on predefined intents and keywords; required frequent manual updates.

Phase 2: LLaMA-2-7B (Instruction-Tuned)

- **Libraries:** Hugging Face Transformers, PyTorch, PEFT (LoRA-based tuning)
- **Trainer:** SFTTrainer with instruction-tuned JSONL dataset
- **Advantages:**
 - Improved generalization
 - More context-aware responses
 - LoRA enabled lightweight fine-tuning

Phase 3: Retrieval-Augmented Generation (RAG) with Mistral-7B-Instruct

- **Architecture:** A RAG pipeline combining a **retriever** and a **generator**.
- **Retriever:** all-MiniLM-L6-v2 sentence transformer for semantic search across center-related documents and FAQs.
- **Generator:** Mistral-7B-Instruct, a lightweight but powerful LLM fine-tuned for conversational tasks.
- **Advantages:**
 - Contextually accurate responses grounded in the center's knowledge base.
 - No dependency on manual intent definitions.
 - Scalable to new documents and knowledge updates without retraining.

Table 1: Backend Technologies

Feature	Purpose
MongoDB	NoSQL database for dynamic content management
Node.js + Express.js	RESTful API creation, middleware, and custom routing
Rasa	Initial rule/intent-based chatbot system
LLaMA-2-7B	Instruction-tuned model for context-aware responses
Mistral-7B-Instruct (RAG)	Generator model for contextual and accurate conversational responses
all-MiniLM-L6-v2	Dense retriever for semantic similarity and document search
Python	Data preprocessing, chatbot pipeline integration, and model handling

2.3 FRONTEND TECHNOLOGIES

The frontend of the project was developed to provide a dynamic, responsive, and user-friendly interface for the web portal. It was designed to showcase the center's infrastructure, projects, faculty, students, and publications, while also integrating the chatbot and admin functionalities.

2.3.1 React.js

The core frontend was built using **React.js**, a component-based JavaScript library that enabled modular development and efficient rendering of UI components.

- Provided a responsive and interactive user interface.
- Allowed reuse of components across multiple sections (e.g., station pages, project cards).
- Facilitated seamless integration with RESTful APIs and chatbot widget.

2.3.2 Tailwind CSS

For styling, Tailwind CSS was used to design a clean and modern interface.

- Utility-first CSS framework ensured faster development.
- Supported responsive layouts optimized for desktops, tablets, and smartphones.
- Provided consistency in design with minimal custom CSS.

2.3.3 Admin Panel

An admin panel was implemented to enable authorized users to manage portal content dynamically.

- Add, edit, and delete details of interns, faculty, and projects.
- Secured access with authentication and role-based permissions.
- Ensured scalability, allowing future extension to manage news, publications, and announcements.

2.3.4 Chatbot Integration

The frontend integrated the **RAG-based chatbot** through a floating widget, ensuring smooth interaction with users.

- Provided contextual answers to FAQs and project-related queries.
- Assisted users in navigating portal content.
- Offers a conversational interface for improved accessibility.

2.3.5 Mobile Responsiveness

The frontend was fully optimized for cross-device compatibility.

- Adaptive grid layouts ensured seamless user experience on desktops, tablets, and mobile devices.
- Responsive images and optimized components enhanced performance.

Table 2: Frontend Technologies

Feature	Purpose
React.js	Component-based UI development, API integration
Tailwind CSS	Utility-first responsive design and styling
Admin Panel	Dynamic content management for interns, faculty, and projects
Chatbot Widget	Seamless integration of RAG-based conversational assistant
Responsive Design	Cross-device optimization (desktop, tablet, mobile)

CHAPTER 3: SYSTEM DESIGN & IMPLEMENTATION

3.1 SYSTEM OVERVIEW

The internship project involved developing a responsive and scalable web portal for the Center of Excellence in Digital Manufacturing (CoE-DM), integrating both frontend and backend systems with a Retrieval-Augmented Generation (RAG) based AI chatbot.

The portal showcases:

- Stations (MIRAC-PC, TRIAC-PC, Assembly, Inspection, Performance Measurement, AS/RS)
- Archives of projects and publications
- Faculty and student involvement

The RAG-based chatbot aids in user navigation and query resolution, providing contextual responses using semantic retrieval and generation. The overall architecture is modular and cloud-ready, supporting scalability for future extensions such as advanced analytics and CMS features.

3.2 TASKS PERFORMED

The internship was conducted over three phases:

- **Phase 1:** Online learning modules and quizzes via FSM-IITD
- **Phase 2:** Implementation of machine learning models and chatbot prototype
- **Phase 3:** Offline project work involving full-stack web and chatbot development

Table 3: Tasks Performed

Task	Description
Frontend Development	Built responsive UI using React.js and Tailwind CSS
Backend Setup	Node.js + Express.js REST APIs with MongoDB Atlas
Chatbot (Final Phase)	RAG-based chatbot using all-MiniLM-L6-v2 as retriever and Mistral-7B-Instruct as generator
Admin Panel	Dynamic content management for interns, faculty, and projects
Data Storage	MongoDB for structured content

3.3 SYSTEM ARCHITECTURE

The system architecture is modular and cloud-ready, supporting frontend-backend decoupling and chatbot integration.

Components:

- **Client:** React/Next.js-based web interface
- **API Server:** Node.js Express-based endpoints
- **Database:** MongoDB Atlas for dynamic content
- **Chatbot Engine:** Retrieval-Augmented Generation (RAG) pipeline combining dense retrieval (all-MiniLM-L6-v2) with generative modeling (Mistral-7B-Instruct).

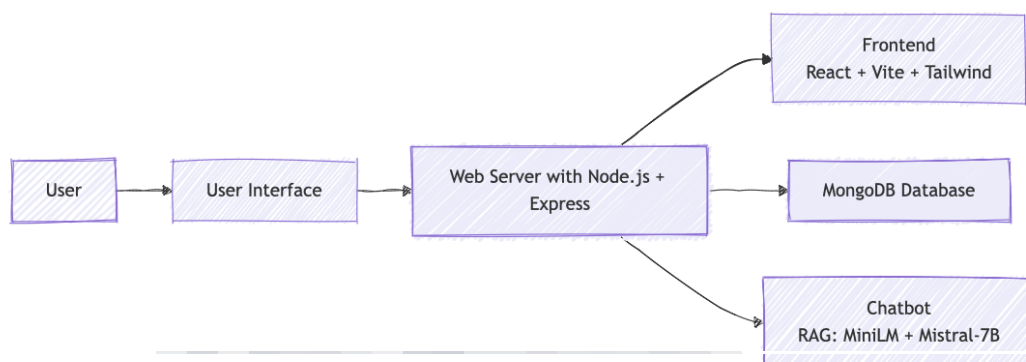


Figure 1: System Architecture

3.4 MODULE DESIGN

3.4.1 Admin Module

- **Responsibilities:**
 - **Dashboard:** View basic analytics and system health.
 - **Station Manager:** Update station descriptions, images, and 3D media.
 - **People Manager:** Maintain records of students, faculty, and contributors.

3.4.2 Chatbot Module

- **Retriever:** all-MiniLM-L6-v2 embedding model for semantic document search.
- **Generator:** Mistral-7B-Instruct LLM for context-aware conversational responses.
- **Integration:** Floating widget on the portal for real-time interaction.
- **Capabilities:**
 - Answer FAQs about CoE-DM
 - Guide users to relevant sections/projects
 - Provide station-specific information

CHAPTER 4: METHODOLOGY AND ALGORITHM

4.1 INTRODUCTION

The methodology adopted for this project follows a hybrid approach, combining modern web development practices with advanced natural language processing (NLP) techniques. The web portal was developed using the **MERN stack (MongoDB, Express.js, React.js, Node.js)**, which provides a scalable, modular, and responsive architecture for managing dynamic content such as stations, projects, faculty, students, and publications.

On top of the web platform, an intelligent **AI chatbot** was integrated to enhance user interaction and provide contextual assistance. The chatbot development was carried out in two phases. The first phase employed **Rasa**, an open-source conversational AI framework, to build an intent-based prototype. In the second phase, a more advanced solution was implemented using **LLaMA-2 (7B)**, fine-tuned with custom instruction datasets and combined with **Retrieval-Augmented Generation (RAG)** for context-aware responses.

The methodology ensures:

- A modular design with clear separation between frontend, backend, and database.
- Scalable backend APIs to support future expansion.
- Intelligent chatbot integration for natural, contextual user interactions.
- Cloud-ready deployment with scope for content management and analytics.

This combination of **MERN stack architecture** and **AI-driven conversational models** enables the system to be both technically robust and user-friendly, ensuring long-term adaptability and scalability.

4.2 METHODOLOGY

The methodology for developing the Interactive Web Portal with Chatbot can be divided into two major components:

1. MERN Stack-based Web Platform
2. AI Chatbot with Retrieval-Augmented Generation (RAG)

4.2.1 MERN Stack Methodology

The web portal was designed and implemented using the MERN stack, which provides a modular and scalable full-stack solution.

- **Frontend (React.js):**
A responsive and interactive user interface was built using React.js with Tailwind CSS. This enables dynamic navigation across different modules such as stations, projects, faculty/student records, and publications.
- **Backend (Node.js + Express.js):**
RESTful APIs were developed to handle CRUD operations for all entities (stations, projects, people). Express.js facilitated routing and communication with the database, ensuring a seamless data flow between client and server.
- **Database (MongoDB Atlas):**
A cloud-hosted NoSQL database was used to manage structured and semi-structured content. This allowed for efficient querying and real-time updates of data such as project archives and student/faculty contributions.
- **Integration (API + Frontend):**
The frontend consumed backend APIs to dynamically fetch and display content. This modularity ensures that any database update is reflected immediately on the portal.

4.2.2 Chatbot Methodology

The chatbot development progressed through three phases, starting from a rule-based prototype to a generative model, and finally an advanced retrieval-driven approach.

- **Phase 1: Rasa Prototype**
An intent-based conversational model was built using Rasa. It relied on rule-based NLU pipelines (SpaCy, DIETClassifier, FallbackPolicy) to handle greetings, FAQs, and station/project queries.
 - Strengths: Quick setup, intent-based responses.
 - Limitations: Required frequent updates to the intent dataset; lacked flexibility and contextual awareness.
- **Phase 2: LLaMA-2 Fine-Tuned Model**
A generative chatbot was built using LLaMA-2 (7B) fine-tuned on instruction datasets. This improved fluency and context-handling in open-ended queries compared to Rasa.
 - Strengths: Generated human-like, context-aware responses.
 - Limitations: Depended only on training data; not grounded in dynamic or factual knowledge.
- **Phase 3: Retrieval-Augmented Generation (RAG)**
As the final enhancement, a RAG-based approach was implemented. Instead of relying solely on predefined intents or a static fine-tuned model, RAG retrieved relevant documents from the knowledge base (vectorized content in MongoDB/FAISS) and passed them to the generator to ensure fact-based, domain-specific answers.
 - Strengths: Combined factual accuracy with generative flexibility.
 - Outcome: Provided up-to-date, reliable responses aligned with portal content.

4.2.3 RAG Architecture

The RAG (Retrieval-Augmented Generation) approach was central to improving chatbot performance. Unlike traditional LLMs, which rely only on pre-trained knowledge, RAG combines retrieval from an external knowledge base with generative capabilities of the model.

Workflow of RAG in the chatbot:

1. User Query → The user enters a query through the floating chatbot widget.
2. Retriever Module → The query is converted into embeddings using a sentence transformer. Relevant documents are retrieved from the MongoDB knowledge base containing station details, project descriptions, and publications.
3. Context Fusion → Retrieved chunks of relevant text are combined with the user query.
4. Generator (LLaMA-2) → The augmented input is passed to the fine-tuned LLaMA-2 model, which generates a context-aware and accurate response.
5. Response Delivery → The response is displayed to the user through the chatbot interface.

This ensures that answers remain accurate, dynamic, and up-to-date, even if the base model has not seen the most recent data.

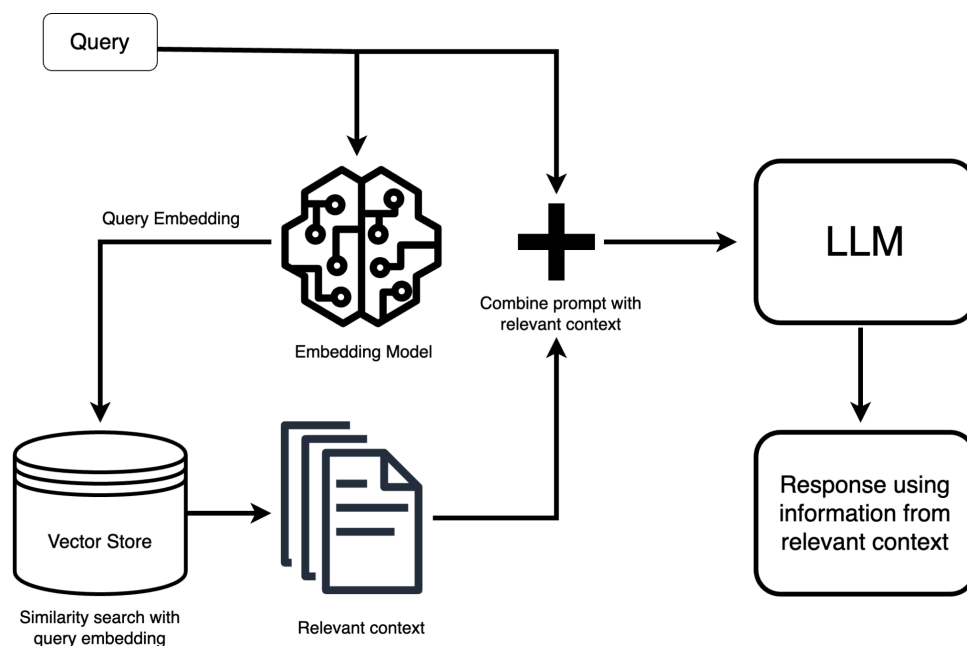


Figure 2: RAG Architecture

4.3 ALGORITHM and AI MODELS

The chatbot and intelligent backend system were built using a combination of **classical NLP pipelines, deep learning architectures, and retrieval-based methods**. The methodology evolved through multiple stages of experimentation to improve accuracy, flexibility, and factual grounding.

4.3.1 Rasa NLP Pipeline (Phase 1)

The initial prototype was developed using the **Rasa Open Source framework**, which employed rule-based NLU pipelines and supervised learning models.

- **Tokenization & Preprocessing**
 - Library: **SpaCy**
 - Function: Split input text into tokens and normalize for processing.
- **LexicalSyntacticFeaturizer**
 - Extracted contextual features like word positions, part-of-speech tags, and dependency relations.
- **DIETClassifier**
 - Model: Transformer-based architecture optimized for intent classification and entity recognition.
 - Input: Token embeddings + features.
 - Output: Predicted **intent** (e.g., greeting, station_query, faq) and extracted **entities** (e.g., station name, project title).
- **Policies Used**
 - **Memoization Policy**: Returns responses for known queries.
 - **TED Policy**: Transformer-based dialogue policy for predicting next action.
 - **Fallback Policy**: Ensures safe handling of out-of-scope queries.

Algorithm Type: Supervised classification + rule-based dialogue management.

4.3.2 LLaMA-2 Fine-Tuned Model (Phase 2)

To overcome the limitations of rule-based systems, a generative AI model was introduced. The **LLaMA-2-7B** (Meta AI) model was selected for its balance of performance and computational efficiency.

- **Model Architecture**:
 - Decoder-only **Transformer** with self-attention.
 - Pre-trained on a large corpus, fine-tuned for **instruction-following** tasks.
- **Fine-Tuning Method**:
 - **Library**: HuggingFace Transformers + PEFT (Parameter-Efficient Fine-Tuning).
 - **LoRA (Low-Rank Adaptation)**: Reduced memory usage by fine-tuning only a small subset of parameters.
 - **Dataset Format**: JSONL, with instruction-response pairs specific to **stations, projects, and FAQs** of the CoE-DM.
- **Training Process**:
 - **Frameworks**: PyTorch + HuggingFace Trainer API.
 - **Trainer Used**: SFTTrainer (Supervised Fine-Tuning).
 - **Hardware**: Google Colab GPU environment.

Algorithm Type: Transformer-based generative language model (instruction-tuned).

4.3.3 Retrieval-Augmented Generation (Phase 3)

In the final phase, **RAG (Retrieval-Augmented Generation)** was integrated to improve factual accuracy. Unlike purely generative models, RAG grounds its answers in external knowledge sources.

- **Core Workflow:**
 - **Query Encoding:** User input converted into embeddings using **Sentence Transformers**.
 - **Document Retrieval:** Relevant passages fetched from a **vector database** (FAISS or MongoDB Atlas Vector Search).
 - **Augmented Prompting:** Retrieved context appended to the user query.
 - **Response Generation:** The generator (baseline LLaMA-2 or other LLM) produces a context-grounded answer.
- **Advantages:**
 - Factual consistency with database content.
 - Scalability as knowledge base can be updated independently of the model.
 - Reduced hallucination in chatbot responses.

Algorithm Type: Hybrid (Information Retrieval + Generative Transformer).

CHAPTER 5: SYSTEM SNAPSHOTS

5.1 DATABASE INTERFACE

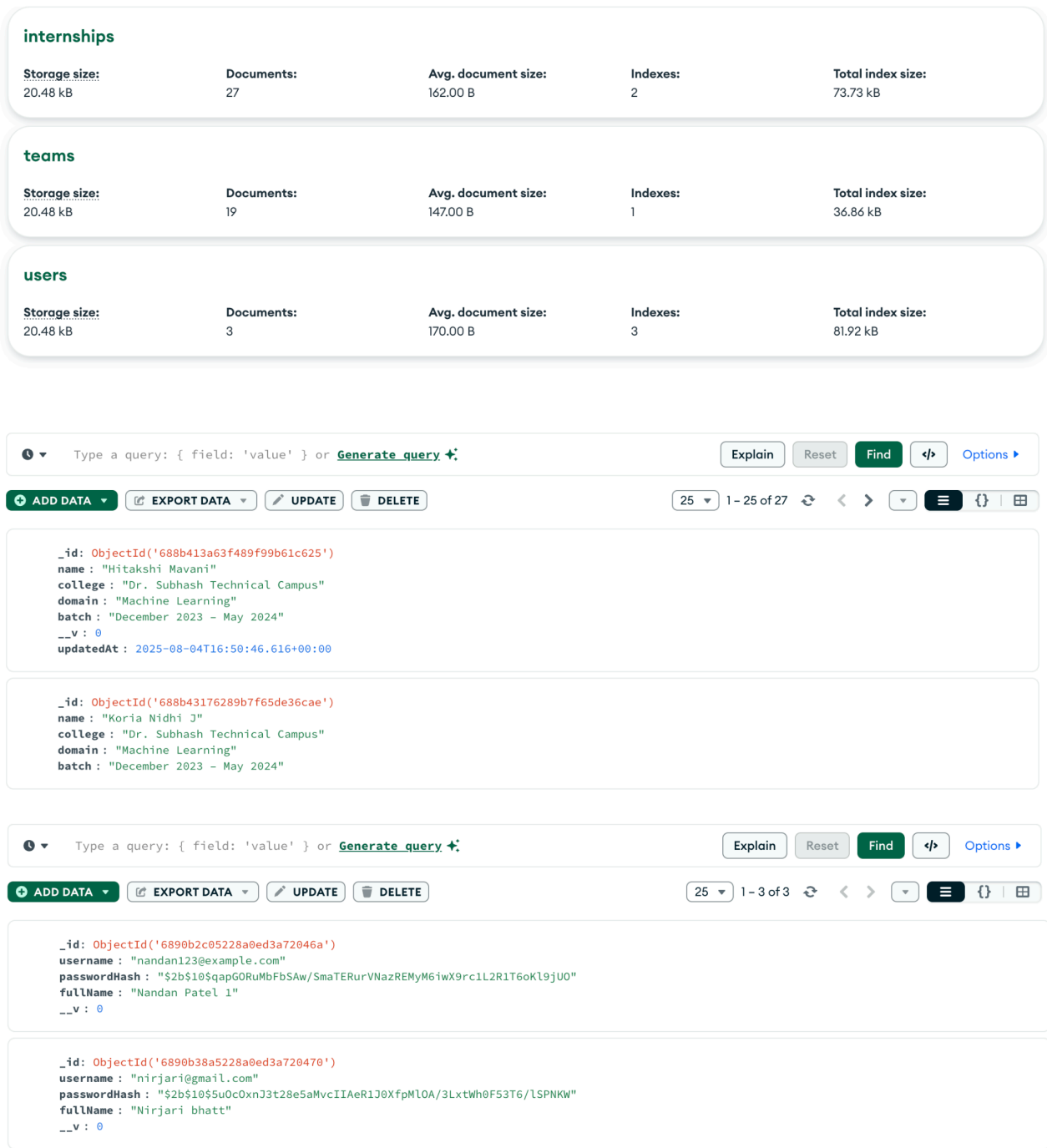


Figure 3: Database collections

5.2 FRONTEND USER INTERFACE



Figure 4: Frontend snapshots

5.3 ADMIN PANEL

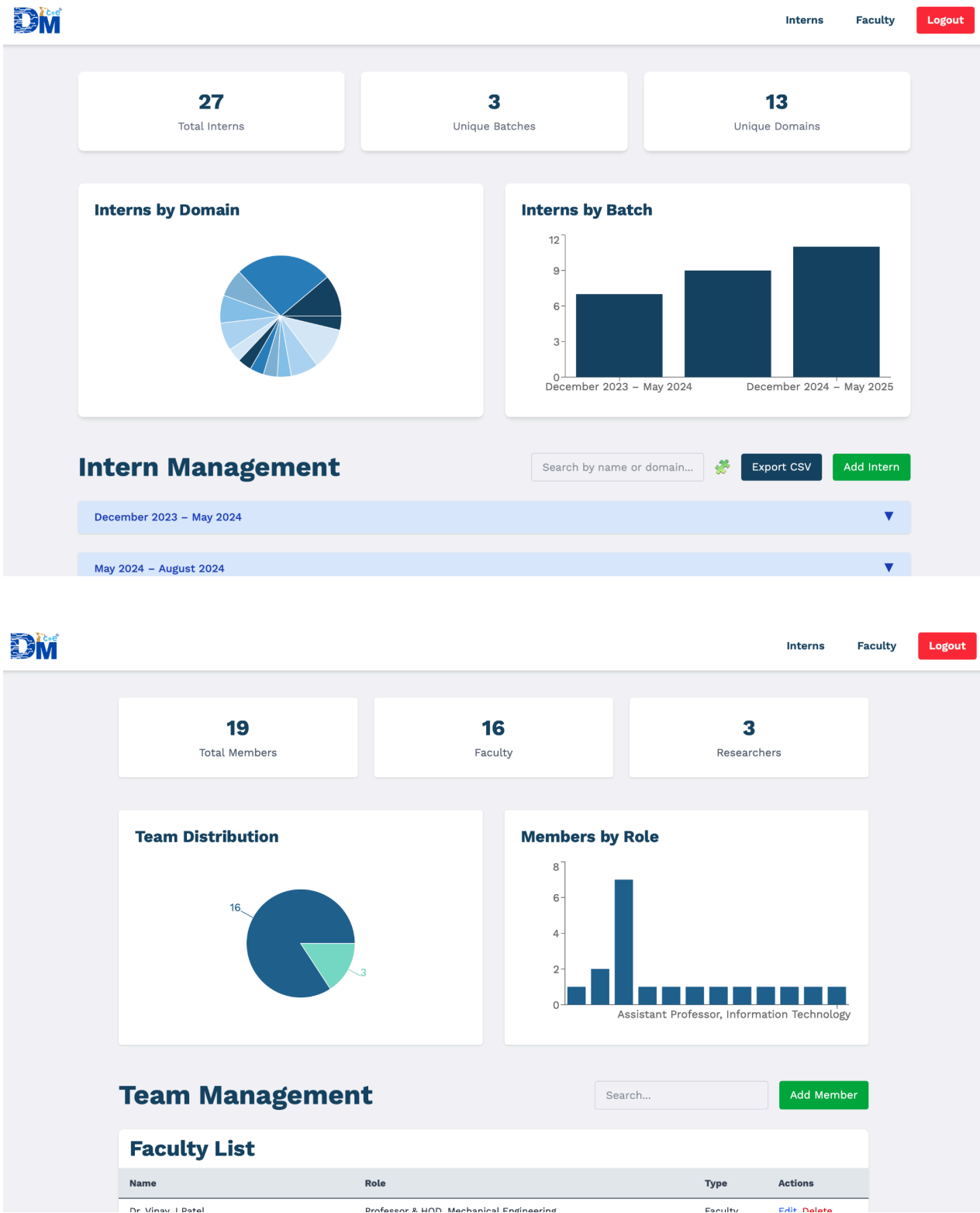


Figure 5: Admin Panel

5.4 CHATBOT INTERFACE

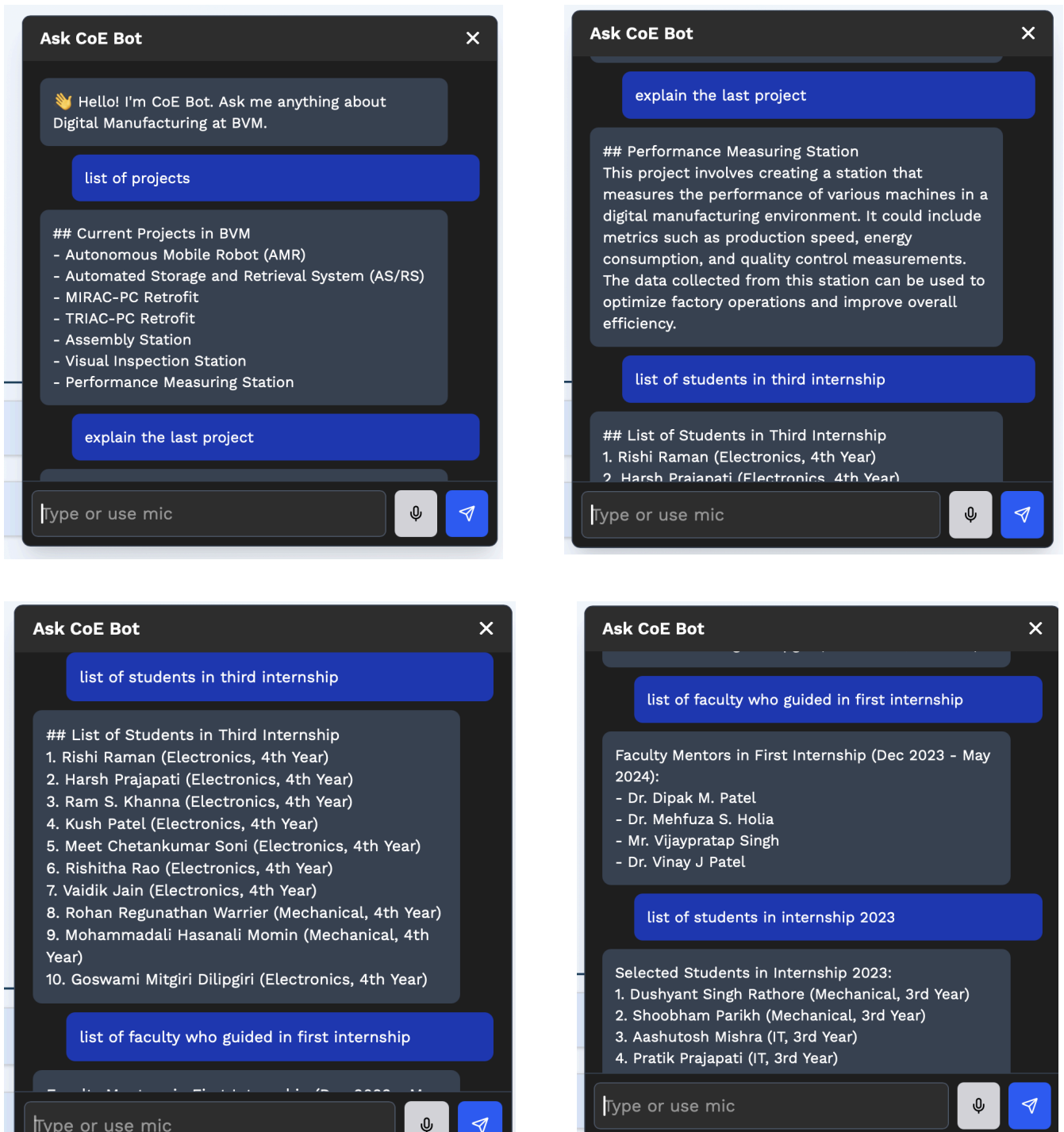


Figure 6: Chatbot Inference

CHAPTER 6: CONCLUSION

Over the span of my 8-week internship, I have successfully contributed to rebuilding and modernizing the organization's digital platform, along with developing and deploying a personalized AI-driven chatbot.

Key Contributions:

- **Web Platform Redevelopment:** Rebuilt and redesigned the organization's website using the MERN stack (MongoDB, Express.js, React.js, Node.js), ensuring a modern, scalable, and responsive user interface.
- **Cross-Platform Compatibility:** Delivered a platform fully optimized for mobile, tablet, and desktop, enabling a seamless user experience across devices.
- **Cloud-Based Media Management:** Integrated AWS S3 for secure, reliable, and scalable storage of audio-visual resources.
- **Chatbot Development (Phase 1):** Built a Rasa-based chatbot to handle FAQs and structured queries, powered by custom intent classification and fallback policies.
- **Hybrid Chatbot System (Phase 2):** Implemented fallback integration with Gemini 1.5 Pro (Google Generative AI) whenever Rasa's intent confidence dropped below 0.3, enabling more context-aware and accurate responses.
- **System Integration:** Unified both chatbot systems through REST API integration, leveraging Gemini's API for intelligent fallback responses.
- **Development & Collaboration Tools:** Employed VS Code for development, Git/GitHub for version control, and Postman for API testing and debugging.

This internship enabled me to gain hands-on experience in full-stack web development, cloud integration, and AI-driven chatbot systems, while contributing to the organization's mission of building an intelligent and scalable digital ecosystem

CHAPTER 7: REFERENCES

[1] Birla Vishvakarma Mahavidyalaya Engineering College (BVM)

Website of BVM Engineering College providing institutional information, departments, academic programs, and resources.

URL: <https://bvmengineering.ac.in>

[2] Hugging Face Blog – Fine-tuning LLaMA 2

An official blog post by Hugging Face explaining the process of fine-tuning Meta’s LLaMA 2 models using Transformers and PEFT.

URL: <https://huggingface.co/blog/llama2#how-to-fine-tune-llama-2>

[3] AutoGPTQ – GitHub Repository

Open-source library for quantizing LLMs efficiently, especially optimized for inference with GPTQ.

GitHub Repository: <https://github.com/AutoGPTQ/AutoGPTQ>

[4] LoRA: Low-Rank Adaptation of Large Language Models

Hu et al., 2021 – A research paper proposing LoRA, a parameter-efficient technique for adapting pre-trained models.

arXiv Link: <https://arxiv.org/abs/2106.09685>

[5] LIMA: Less is More for Alignment

Zhou et al., 2023 – A research paper demonstrating that high-quality responses from LLMs can be achieved with a small amount of instruction tuning data.

arXiv Link: <https://arxiv.org/abs/2312.12148>

[6] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela – A seminal paper introducing **RAG**, which combines retrieval with generative models for improved factuality and knowledge-intensive tasks.

arXiv Link: <https://arxiv.org/abs/2005.11401>